

Figure 14.16
A two-node cluster with one master

Now, let's throw a wrench in the works. Let's say Node B dies due to a hardware problem. Because Node A is the master, it continues working to service queries with one node: we effectively have a one-node cluster while we wait for another Node B to boot up to join the cluster.

Here's where things can get tricky. While Node B is booting up, suppose network connectivity is severed, making Node B unable to see the existence of Node A. This leads to Node B assuming a `master` role because it thinks there's no master in the cluster, even though Node A exists as the master. This leads to a *split-brain* situation (see figure 14.17).

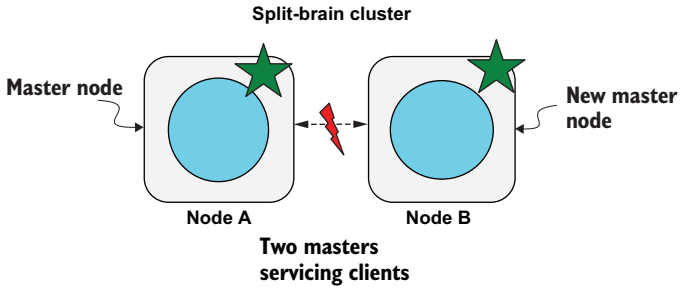


Figure 14.17
Split-brain cluster: a cluster with two masters

The two nodes do not communicate due to network problems, so they work happily as part of the cluster. But because both nodes are masters, any requests coming to either one are carried out only by the receiving node. However, data in one node is not visible to the other node, which raises data discrepancies. This is one of the reasons to have at least three master-eligible nodes in a cluster. Having three such nodes avoids split-brain cluster formation.

14.6.6 Dedicated master nodes

Because a node can be assigned multiple roles, it's not a surprise to see a cluster with 20 nodes where all nodes are performing all roles. There is no harm in creating this type of cluster architecture; however, such a setting works only for lightweight cluster